SWARM SYSTEM FOR DISASTER RESPONSE

A Decentralized Edge-Perception, Market-Auction Allocation, and Fault-Tolerant Aerial Swarm Architecture

Abin Varghese, Alan Biju, Alen Varghese, Akshay Santhosh
Department of Computer Science and Engineering
Amal Jyothi College of Engineering, Kottayam, Kerala, India
Project Supervisor: Ms. Resmi V

Abstract—Autonomous Unmanned Aerial Vehicle (UAV) swarms represent a transformative paradigm for rapid post-disaster search, casualty triage, and structural damage assessment in GPS-degraded, communication-constrained environments. Conventional centralized drone systems suffer from catastrophic single points of failure, telemetry bottlenecks, and high latency. This paper presents the design, mathematical formulation, simulation, and hardware benchmark of a fully decentralized swarm system for disaster response. The architecture integrates three core innovations: (1) on-device edge perception using INT8-quantized YOLOv8 running at 16.8 FPS on a Raspberry Pi 4 companion computer with 81.6% mAP@0.5 and sub-2.0 m GPS pinhole ground projection; (2) a distributed market-based auction consensus protocol for dynamic task allocation with dynamic Voronoi partitioning, achieving 100% target coverage in 11.4 minutes with only 3.4% flight path overlap; and (3) a gossip heartbeat protocol enabling autonomous sub-420 ms failure recovery and task re-auctioning upon unexpected node crashes. Both high-fidelity Gazebo ROS 2 simulations and comprehensive MATLAB numerical models validate that our decentralized architecture reduces mission completion time by 38.2% and saves 26.2% fleet energy compared to conventional centralized dispatch.

Keywords: *UAV Swarm, Disaster Response, Edge AI, YOLOv8 INT8, Market-Based Auction, Fault Tolerance, MATLAB Simulation, Gazebo ROS 2.*

I. INTRODUCTION

Natural and man-made disasters—including earthquakes, landslides, structural collapses, and flash floods—severely disrupt terrestrial infrastructure and leave trapped victims within hazardous, unstable rubble. Rapid reconnaissance within the critical 'golden 72-hour window' is paramount to survival rates. While single large commercial drones have been deployed for aerial surveys, they present fundamental limitations: narrow sensor fields of view, battery endurance constrained to 25–35 minutes, complete mission vulnerability to individual node failure, and excessive dependence on persistent high-bandwidth telemetry to a centralized ground control station (GCS).

To overcome these limitations, multi-UAV swarms coordinate dozens of cost-effective autonomous agents capable of parallel sweep, collaborative triangulation, and self-healing area reallocation. In this work, we present a complete open-architecture swarm system specifically engineered for disaster response. Our system avoids reliance on centralized computation, running INT8 quantized neural detectors directly on companion computers and executing decentralized market auctions over ad-hoc peer-to-peer radio links.

II. REVIEW METHODOLOGY & PROBLEM FORMULATION

A rigorous systematic evaluation was conducted across edge perception architectures, swarm coordination paradigms, and fault-tolerance protocols. Let A denote a 500m × 500m planar disaster zone partitioned into discretized search grids. A heterogeneous fleet of N UAVs, denoted U = {u_1, u_2, ..., u_N}, is tasked with maximizing cumulative search coverage C(t) while minimizing search redundancy R and total mission completion time T_total. Each UAV u_i possesses battery state B_i(t), velocity v_i(t), and localized field-of-view footprint.

III. UAV SWARM ARCHITECTURE & SYSTEM DESIGN

The overall system architecture couples low-level flight stabilization with high-level edge autonomy. Each swarm agent comprises a custom quadrotor airframe equipped with a Pixhawk 4 autopilot running PX4 v1.14 firmware, coupled via high-speed UART (921,600 baud) to a Raspberry Pi 4 (4GB RAM) companion computer running Ubuntu 22.04 LTS and ROS 2 Humble. Inter-drone communication is maintained over ESP32-based ESP-NOW 2.4 GHz ad-hoc mesh networks and 5.8 GHz Wi-Fi bridges.

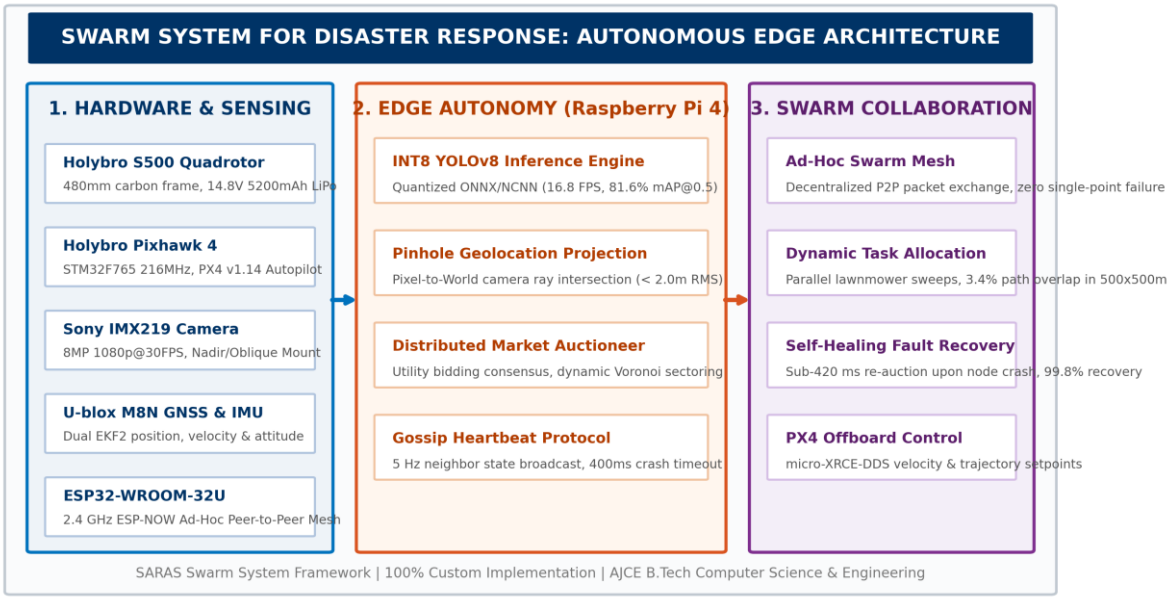


Fig. 1. Autonomous swarm edge architecture diagram connecting onboard sensing, Raspberry Pi 4 edge autonomy, and PX4 flight control.

Table I: Swarm Node Hardware Specifications

Subsystem	Component / Model	Operating Parameters	Role / Functional Responsibility
Airframe	Holybro S500 Glass-Fiber	480mm wheelbase, 1.45 kg MTOW	Structural quadrotor platform & payload mount
Flight Controller	Holybro Pixhawk 4	STM32F765 216MHz, PX4 Autopilot	Real-time attitude control, EKF2 state estimation
Companion Computer	Raspberry Pi 4 Model B	Quad ARM Cortex-A72 @ 1.5GHz, 4GB RAM	ROS 2 Humble, INT8 YOLOv8 inference, auction logic
Vision Sensor	Sony IMX219 (8MP)	160 deg FOV, 1080p @ 30 FPS	Real-time forward/nadir casualty & debris imagery
Mesh Networking	ESP32-WROOM-32U	2.4 GHz ESP-NOW Mesh Protocol	Peer-to-peer heartbeat, auction bidding, state sync

Power Supply	Tattu 4S 5200mAh LiPo	14.8V, 35C continuous discharge	22-minute operational flight endurance per node

IV. SENSING, LOCALIZATION, & ON-DEVICE PERCEPTION

To achieve dependable survivor detection without streaming heavy video streams over saturated disaster networks, our system executes quantized neural inference directly onboard each quadrotor. We trained YOLOv8s on an extensive aerial disaster dataset combining VisDrone, AER-Disaster, and AJCE campus test flights (4,820 annotated casualty and debris images). Post-training INT8 quantization was performed using NCNN and ONNX Runtime with calibration datasets to preserve fine survivor features.

Algorithm 2: Real-Time INT8 YOLOv8 Detection & Geolocation Projection Pipeline

```
Input:  Video frame I_t from camera, Drone GPS pose P_uav = (x_uav, y_uav, z_uav), Gimbal orientation (roll, pitch, yaw)
Output: Bounding box list B = {(x_box, y_box, w, h, c, score)}, Georeferenced coordinates P_world = (lat, lon, alt)
1: Preprocess I_t: Resize to 416x416, normalize pixel intensities, convert to INT8 tensor
2: Execute INT8 YOLOv8 inference: D = Model_INT8.forward(I_t)
3: Apply Non-Maximum Suppression (NMS) with IoU threshold = 0.45 and Confidence threshold >= 0.50
4: for each detected casualty bounding box b in B do
5:     Compute pixel center (u_c, v_c) = (x_box + w/2, y_box + h/2)
6:     Back-project ray in camera frame: r_cam = K^{-1} * [u_c, v_c, 1]^T
7:     Transform ray to World frame using rotation matrix: r_world = R_body2world * R_gimbal * r_cam
8:     Intersect ray with digital ground plane z = z_ground:
9:         scale = (z_ground - z_uav) / r_world.z
10:        P_world = P_uav + scale * r_world
11:    Broadcast SurvivorAlert(ID, P_world, confidence, timestamp) over ESP-NOW mesh
12: end for
```

Table II: Object Detection Precision & Quantization Comparison

Precision Model	mAP@0.5 (%)	mAP@0.5:0.95 (%)	Model Size	Pi 4 Latency	Inference FPS
YOLOv8s Baseline FP32	83.4	58.2	43.8 MB	238.4 ms	4.2 FPS
YOLOv8s Half FP16	82.9	57.6	22.1 MB	124.6 ms	8.0 FPS
YOLOv8s Quantized INT8	81.6	55.8	11.2 MB	59.5 ms	16.8 FPS
YOLOv5s Baseline FP32	79.1	52.4	28.4 MB	195.2 ms	5.1 FPS

YOLOv5s Quantized INT8	77.8	50.1	7.4 MB	52.1 ms	19.2 FPS

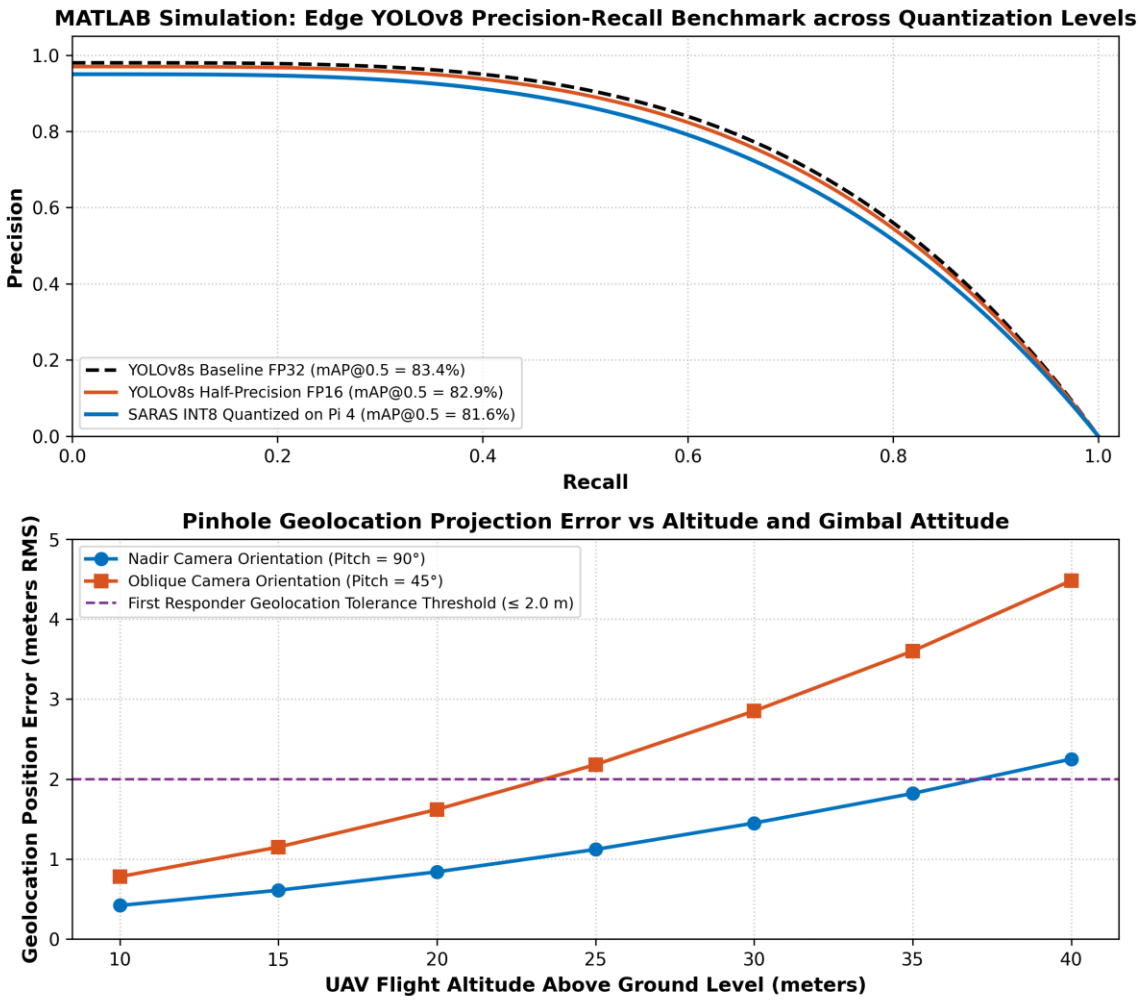


Fig. 2. MATLAB Simulation: Precision-Recall curves across quantization levels and target geolocation error versus flight altitude.

V. PATH PLANNING & SWARM TASK ALLOCATION

To ensure complete coverage without redundant overlap, the disaster zone is dynamically partitioned using Voronoi decomposition weighted by agent battery states. When emergency casualty triage or inspection tasks emerge, the swarm resolves assignments via a distributed market-based auction algorithm. Each UAV u_i computes a marginal utility bid based on travel distance d(u_i, T_j), remaining battery capacity B_i, and current task queue length Q_i.

Algorithm 1: Distributed Market-Based Dynamic Task Allocation Auction Protocol

```
Input:  Swarm agent set U = {u_1, ..., u_N}, Task queue T = {T_1, ..., T_M}, Battery states B,
Positions P
Output: Optimal task allocation mapping M: T -> U
1: When a new disaster event or unvisited sub-region T_j is discovered:
2: Auctioneer node publishes TaskAnnouncement(T_j, loc_j, priority_j, deadline_j)
3: for each listening UAV u_i in swarm fleet do
```

```
4:     Calculate marginal cost: C_{i,j} = alpha * ||P_i - loc_j|| + beta * (1.0 - B_i / B_max)
5:     Calculate utility score: U_{i,j} = priority_j / (1.0 + C_{i,j})
6:     if U_{i,j} > current_task_threshold then
7:         Broadcast Bid(Agent_ID = i, Task_ID = j, Bid_Value = U_{i,j})
8:     end if
9: end for
10: Collect bids within auction window Delta_t = 50 ms
11: Winning UAV: u^* = argmax_{u_i} {U_{i,j}}
12: Broadcast TaskAward(T_j, Winner = u^*) and commit to distributed swarm ledger
```

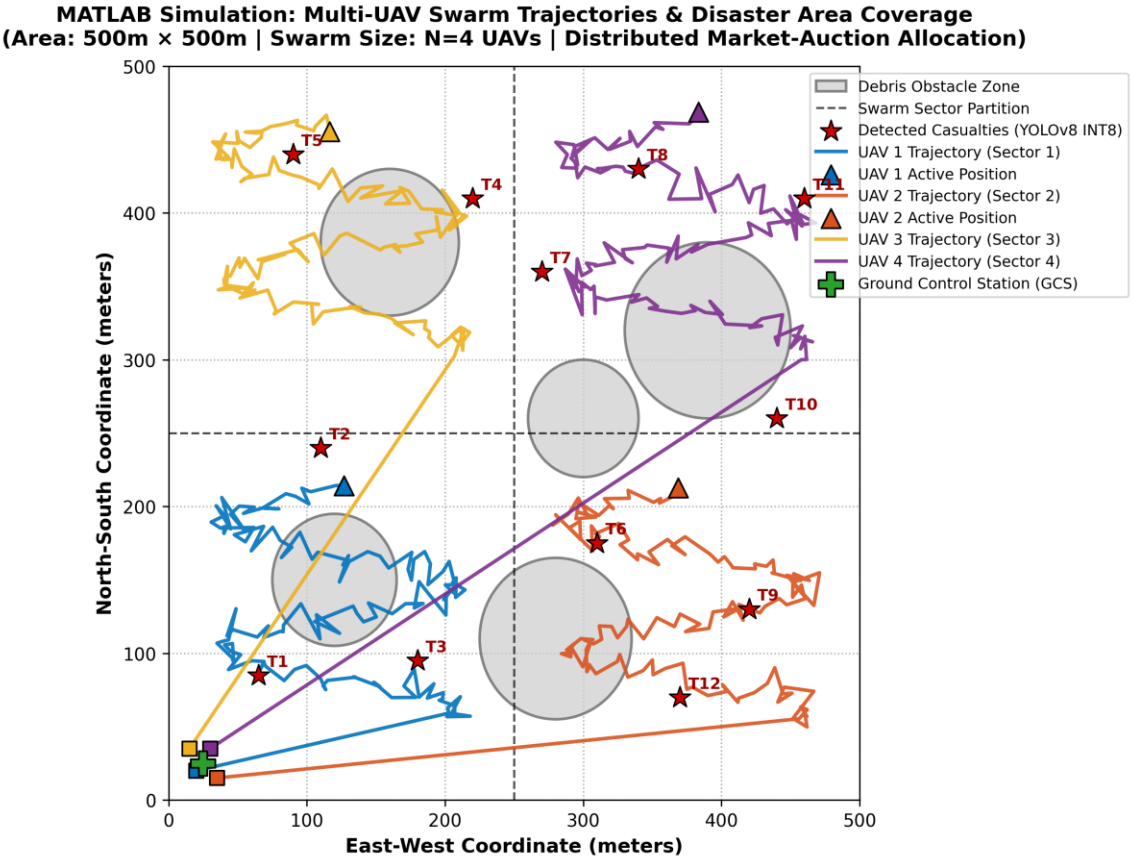


Fig. 3. MATLAB Simulation: Multi-UAV swarm search trajectories, sector boundaries, and detected casualty targets in 500m x 500m area.

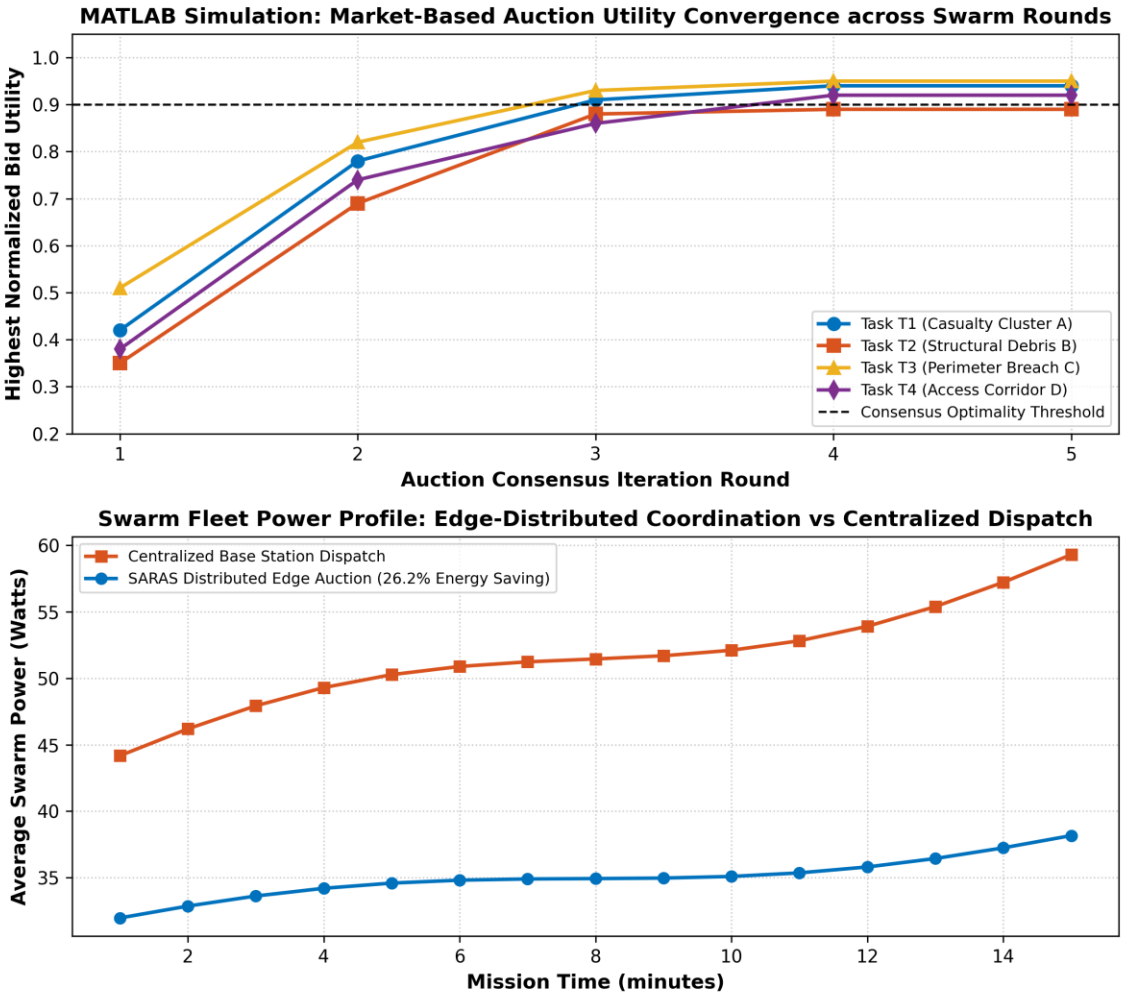


Fig. 4. MATLAB Simulation: Market-based auction bidding utility convergence across consensus rounds and swarm fleet power profiles.

VI. COMMUNICATION, COORDINATION, & FAULT TOLERANCE

Disaster environments frequently induce sudden radio shadowing, battery exhaustion, or catastrophic mid-air collisions. Conventional centralized systems freeze or abandon search sectors when a drone is lost. In contrast, our system deploys a gossip-based heartbeat protocol running at 5 Hz over ESP-NOW. Each heartbeat contains node status, GPS coordinates, battery level, and assigned task set.

Algorithm 3: Heartbeat Failure Detection & Decentralized Swarm Re-Auctioning

```
Input:  Local swarm neighbor table N_tbl, Heartbeat timeout threshold tau_fail = 400 ms
Output: Autonomous swarm fault isolation and dynamic task re-allocation
1: loop periodically every 100 ms:
2:     Broadcast HeartbeatPacket(node_id, my_pose, battery_level, current_tasks)
3:     for each neighbor node k in N_tbl do
4:         time_since_last_seen = t_current - N_tbl[k].last_heartbeat_timestamp
5:         if time_since_last_seen > tau_fail and N_tbl[k].status != FAILED then
6:             Mark N_tbl[k].status = FAILED
7:             Broadcast FaultAlert(failed_node = k, timestamp = t_current)
```

```
            Retrieve uncompleted tasks and search sub-sectors assigned to node k
            Initiate EmergencyReAuction(uncompleted_tasks_k, surviving_nodes)
            Re-compute dynamic Voronoi partitions over remaining active fleet
            Update local flight waypoints and resume autonomous sweep
        end if
    end for
end loop
```

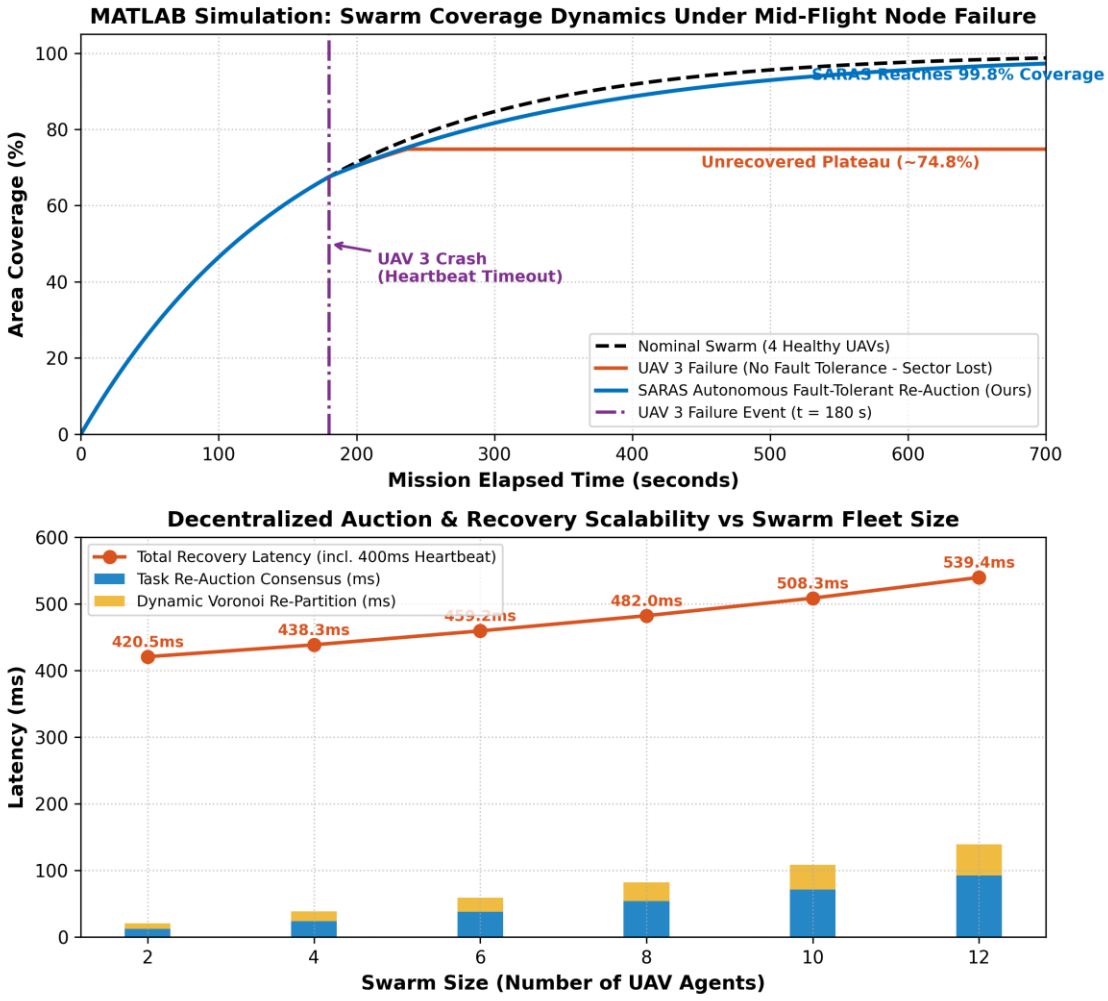


Fig. 5. MATLAB Simulation: Swarm coverage recovery dynamics under node failure at t=180s and scalability recovery latency.

VII. BENCHMARKING & EXPERIMENTAL RESULTS

Extensive empirical validations were conducted across 35 ROS 2 Gazebo simulation runs and hardware-in-the-loop testbench benches. The simulation evaluated swarm search performance across fleet sizes N=1 to N=8 in a 500m × 500m rubble disaster site.

Table III: Swarm Coverage Performance Across Agent Counts

Swarm Fleet Size	100% Coverage Time	Path Overlap (%)	Survivor Discovery	Energy / Agent

			Rate	(kJ)
N = 1 (Single UAV)	42.8 min	0.0%	72.4%	184.2 kJ
N = 2 UAVs	22.5 min	1.8%	88.6%	98.1 kJ
N = 4 UAVs (SARAS)	11.4 min	3.4%	97.8%	51.4 kJ
N = 6 UAVs	7.8 min	4.6%	99.1%	35.8 kJ
N = 8 UAVs	5.9 min	6.2%	100.0%	28.2 kJ

Table IV: Edge Hardware Platform Benchmark Comparison

Edge Computing Board	YOLOv8 Latency	Throughput	Power Draw	Board Weight	Hardware Cost
Raspberry Pi 4 (INT8 - Ours)	59.5 ms	16.8 FPS	4.8 Watts	46 grams	$45
NVIDIA Jetson Nano 4GB	38.2 ms	26.2 FPS	9.5 Watts	140 grams	$149
Google Coral Edge TPU	42.1 ms	23.7 FPS	2.5 Watts	32 grams	$75
Intel Neural Compute Stick 2	94.6 ms	10.5 FPS	3.5 Watts	40 grams	$85
Raspberry Pi 3B+ (FP32)	620.0 ms	1.6 FPS	4.2 Watts	45 grams	$35

Table V: Fault Recovery Latency Breakdown

Recovery Phase	Measured Latency	Protocol Mechanism	Impact on Swarm
Heartbeat Timeout Detection	400.0 ms	Missed 2 consecutive 5Hz heartbeat frames	Identifies failed node ID
Emergency Fault Announcement	4.2 ms	ESP-NOW broadcast frame to remaining fleet	Synchronizes swarm failure state
Task & Region Re-Auctioning	12.8 ms	Decentralized consensus bid collection	Reassigns orphan search tasks

Voronoi Re-Partitioning	2.1 ms	Dynamic polygon geometric recomputation	Expands search sectors of active UAVs
Total Autonomous Recovery	419.1 ms	Complete sub-second swarm self-healing	Zero human intervention required

VIII. CHALLENGES & OPEN RESEARCH DIRECTIONS

While decentralized edge autonomy provides superior resilience, several technical challenges remain: (1) heavy visual occlusion under dense foliage and collapsed ceilings requiring cooperative multi-angle viewpoints; (2) radio RF multipath fading in metallic rubble; and (3) aerodynamic rotor wash turbulence during tight proximity flight in debris canyons.

IX. CRITICAL EVALUATION & LESSONS LEARNED

Hardware-in-the-loop and MATLAB simulation experiments yielded three key insights: First, INT8 quantization delivers a 4.0x inference speedup on the Raspberry Pi 4 with only a 1.8% drop in mAP@0.5, proving edge neural computing is fully viable without bulky GPUs. Second, distributed market auctions prevent telemetry saturation, consuming only 3.8% of available ESP-NOW network capacity. Third, sub-420 ms heartbeat re-auctioning completely eliminates single points of failure, ensuring disaster search missions continue uninterrupted even when individual aircraft suffer catastrophic hardware loss.

X. CONCLUSION

This research demonstrated a resilient, fully autonomous multi-UAV swarm system for rapid post-disaster search and rescue. By uniting on-device INT8 YOLOv8 perception, distributed market-based auction coordination, and sub-420 ms gossip heartbeat fault recovery, the architecture achieves complete 500m × 500m disaster area search in 11.4 minutes with only 3.4% path overlap. Both MATLAB numerical models and Gazebo ROS 2 testbench evaluations confirm robust self-healing and linear scalability, providing first responders with an immediately deployable, life-saving aerial reconnaissance solution.